实验报告

姓名：陈秀萍

学号：2026001406

实验概述

本次实验包括四部分：使用 RcppEigen 完成灰度图像的固定秩近似；利用稀疏矩阵求解高斯过程插值；构建 measError R 包并比较串行与多进程并行模拟；使用 OpenMP 并行实现图像卷积。各程序均从作业目录运行，图像实验使用同一张 JPEG 图片。报告重点记录实现方法、数值验证、图像结果和运行时间。

运行环境

项目	配置
运行平台	RStudio Server，Ubuntu 24.04.1，ARM64 aarch64
R	R 4.6.1
编译器	GCC 13.3.0
主要扩展包	Rcpp 1.1.2；RcppEigen 0.3.4.0.2；RcppArmadillo；foreach；doRNG；doParallel；jpeg
并行配置	多进程使用 1、2、4 个 worker；OpenMP 使用 1、2、4 个线程
处理器	Apple M5，Docker 容器可见 10 个逻辑 CPU

1 图片低秩压缩

1.1 方法与实现

灰度图像可以表示为像素值位于 0 到 1 之间的矩阵。对图像矩阵进行奇异值分解后，保留最大的 K 个奇异值及其对应的左右奇异向量，可以得到秩为 K 的近似。

$$A = U \Sigma V^{\top}$$

$$\widehat{A}_K = \sum_{i=1}^{K} \sigma_i u_i v_i^{\top}$$

C++ 函数 fixedRankApprox 使用 Eigen 的 JacobiSVD，并采用 ComputeThinU 和 ComputeThinV 只计算薄矩阵形式。重构完成后，对每个像素进行截断，使输出仍位于 0 到 1 之间。函数同时检查输入矩阵非空、元素有限，并要求 K 位于 1 与矩阵较小维数之间。

1.2 数值验证

测试包括对角矩阵、3×2 矩阵、1×1 矩阵以及自选的 5×7 随机矩阵。随机矩阵分别取 K=1、3、5，并与 R 的 svd 函数结果逐元素比较。所有测试均保持原矩阵尺寸，输出像素位于 0 到 1 之间；与 R 参考结果的最大绝对误差均小于 10^{-8}，满秩情况下也能在 10^{-8} 精度内恢复输入矩阵。

1.3 图像结果与存储量

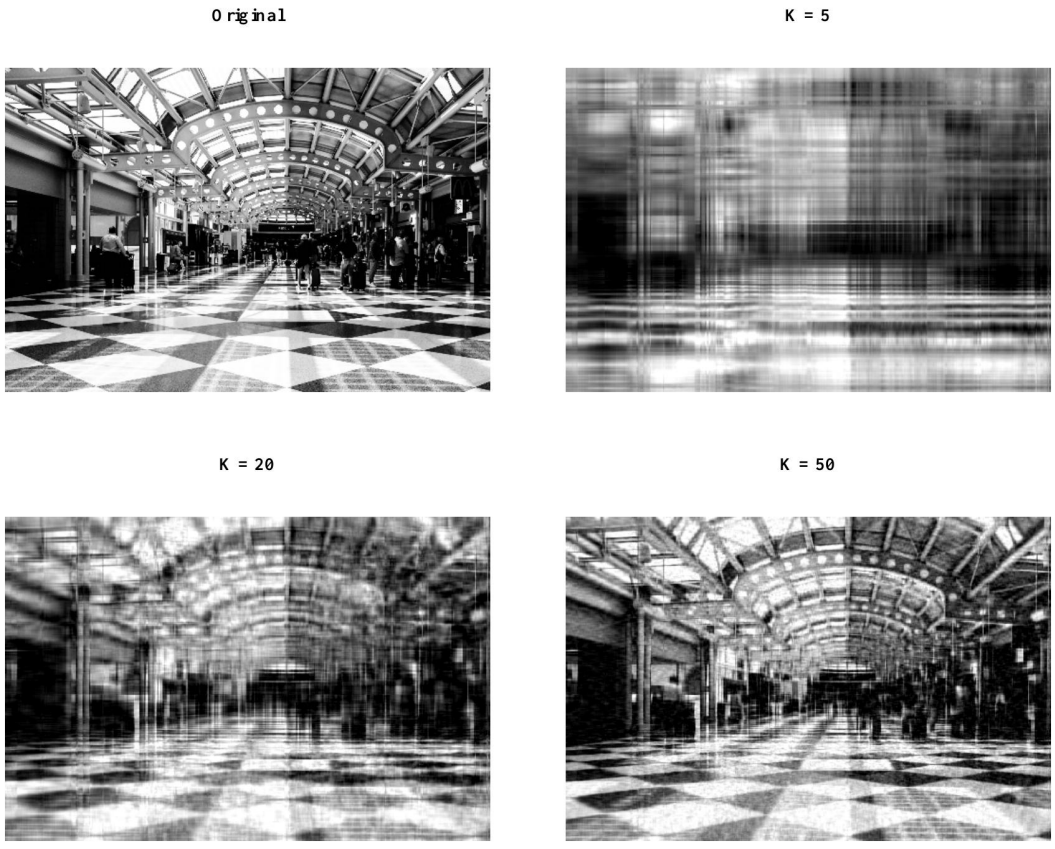
原始图片尺寸为 1365×2048。脚本按步长 4 进行缩放，实际用于 SVD 的灰度矩阵尺寸为 342×512，共 175104 个像素。相对 Frobenius 误差和低秩因子存储比例分别定义为

$$E_K = \frac{\| A - \widehat{A}_K \|_F}{\| A \|_F}$$

$$R_K = \frac{K(m+n+1)}{mn}$$

K	相对 Frobenius 误差	因子存储数	原图像素数	存储比例
5	0.33818	4275	175104	2.4414%
20	0.23353	17100	175104	9.7656%
50	0.15482	42750	175104	24.4141%

图 1 原图及 K=5、20、50 时的低秩近似结果



当 K=5 时，图像主要保留大范围的明暗结构，建筑边缘和地面纹理较模糊；K 增加到 20 后，拱顶和地面线条逐渐清晰；K=50 时，主要场景和细节已经较容易辨认。误差随 K 增大而下降，但因子存储量近似按 K 线性增长。本实验比较的是低秩因子的元素数量，而不是 JPEG 文件大小。

2 高斯过程插值

2.1 模型与稀疏求解

在 D 个规则网格点上，二阶差分矩阵 L 描述曲线的局部平滑性，观测矩阵 A 用于选出带噪观测位置。给定平滑参数和观测噪声方差，后验精度矩阵与均值满足

$$Q = \lambda^2 L^{\top} L + \frac{A^{\top} A}{\sigma_y^2}$$

$$Q\mu = \frac{A^{\top} y}{\sigma_y^2}$$

程序以 Eigen 的 SparseMatrix 构造 L、A 和 Q，再使用 SimplicialLLT 对 Q 进行稀疏 Cholesky 分解。均值通过求解线性方程得到；后验标准差由 Q 的逆矩阵对角元素开平方得到。这样在形成精度矩阵和求解均值时利用了矩阵的稀疏结构。

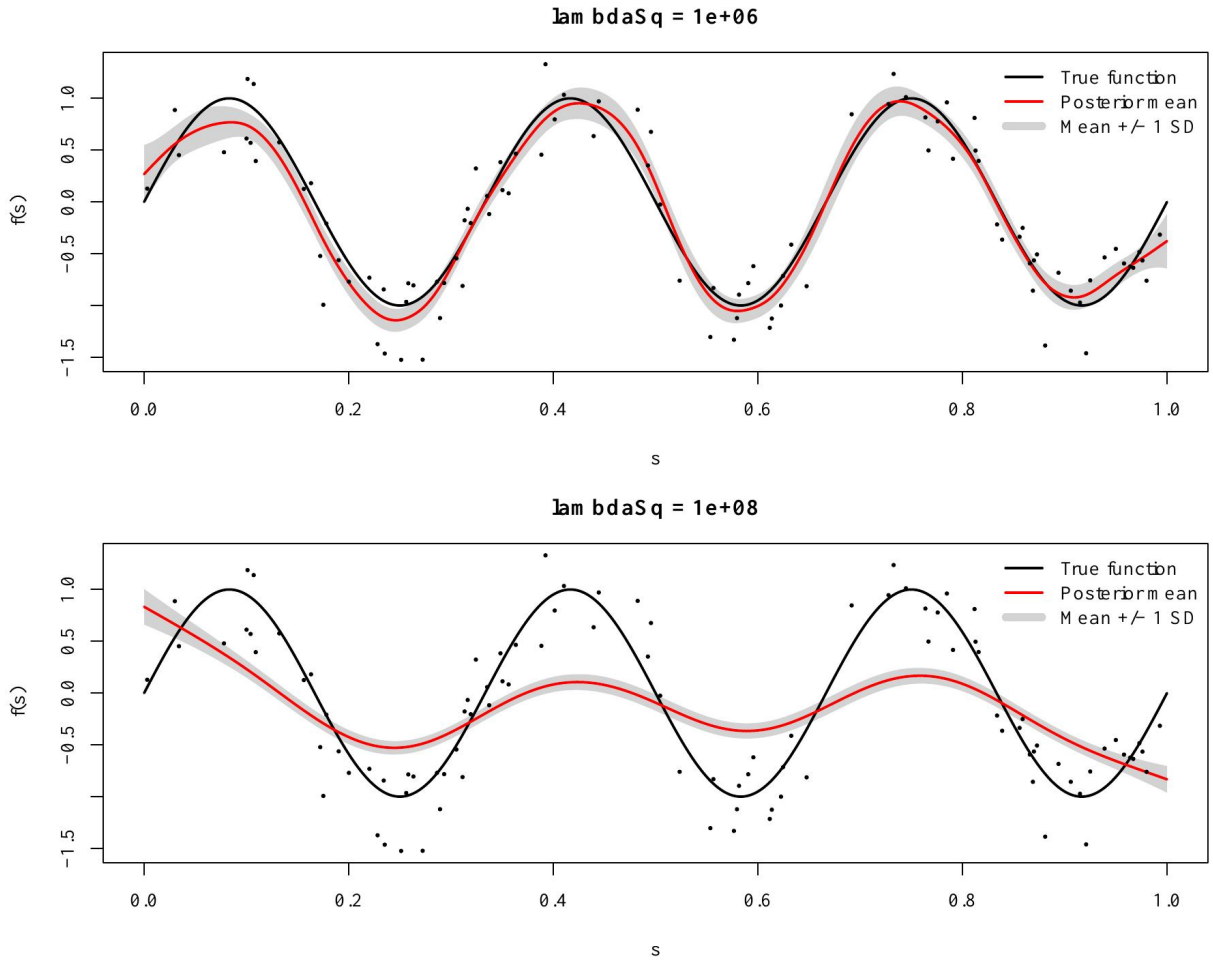
2.2 验证设置

小规模验证取 D=6、3 个观测点、lambdaSq=10 和 sigmaSq_y=0.2。C++ 输出为 6×2 矩阵，第一列为后验均值，第二列为后验标准差。输出元素均为有限值，标准差非负；与 R 稠密矩阵求解结果比较，均值和标准差的最大绝对误差均小于 10^{-8} 。

2.3 参数比较

正式实验取 D=1000、观测点数 n=100、噪声方差 sigmaSq_y=0.1，并固定随机种子 2026。两次拟合使用完全相同的观测位置和观测值，仅将 lambdaSq 分别设为 10^6 和 10^8 。

图 2 相同观测数据下不同 lambdaSq 的高斯过程插值结果



lambdaSq=10^6 时，后验均值能够较好地跟随真实函数的三次周期变化；lambdaSq=10^8 时，对二阶差分的惩罚更强，后验均值明显更平滑，振幅也被压缩，出现欠拟合。由此可见，lambdaSq 越大，对曲率变化的限制越强；参数过大虽然提高了平滑程度，但可能损失真实的局部波动。

3 R 包构建与并行模拟

3.1 measError 包结构

measError 包使用 Rcpp 编写三个接口函数，并通过 R 包安装流程编译。包中包含 DESCRIPTION、NAMESPACE、Rcpp 导出文件、帮助文档和 smoke test。安装后 library(measError) 能够正常加载，基本测试通过。

函数	作用
getData	生成潜变量、两个带误差代理变量以及响应变量
naiveRegression	忽略测量误差，直接对 Y 关于 W 做最小二乘回归
correctedRegression	使用第二个代理变量 Q 作为 W 的工具变量进行校正

3.2 模型与估计量

模拟模型中，W 和 Q 是潜变量 X 的两个独立带噪代理变量，响应变量由 X 的线性模型生成。

$$X_i \sim N(0,1), \quad W_i = X_i + U_i, \quad Q_i = X_i + V_i, \quad Y_i = \beta_0 + \beta_1 X_i + E_i$$

其中，U_i、V_i和E_i分别表示测量误差和随机扰动，均值为零，并与潜变量 X_i独立，两个代理变量的测量误差相互独立。具体误差分布及标准差设置按照模拟程序中的设定执行。

朴素估计直接使用 W，因此测量误差会使斜率向零衰减。校正估计利用 Q 与 W 共享潜变量 X、但测量误差相互独立的性质。两种斜率估计量为

$$\widehat{\beta}_1^{\text{naive}} = \frac{\text{Cov}(W, Y)}{\text{Var}(W)}$$

$$\widehat{\beta}_1^{\text{corrected}} = \frac{\text{Cov}(Q, Y)}{\text{Cov}(Q, W)}$$

3.3 模拟设置与可重复性

实验设置为 n=400、beta0=1、beta1=2、sigmaE=0.2，共重复 B=1000 次。sigmaU 从 0.4 到 0.6 的均匀分布生成，sigmaQ 从 0.1 到 0.2 的均匀分布生成，并在 1000 次模拟中保持不变。每次任务重新生成观测数据，在同一组数据上计算 naive 和 corrected 估计。误差指标为

$$e = \| \widehat{\beta} - \beta \|_2$$

串行与并行版本均通过 doRNG 分配相同的任务级随机数流。串行程序重复运行的结果完全一致，并行程序在 1、2、4 个 worker 下也与串行结果逐项一致，说明实验具有可重复性。

3.4 估计误差与运行时间

方法	平均误差	中位误差
Naive	0.4020659	0.4006084
Corrected	0.06367544	0.05906190

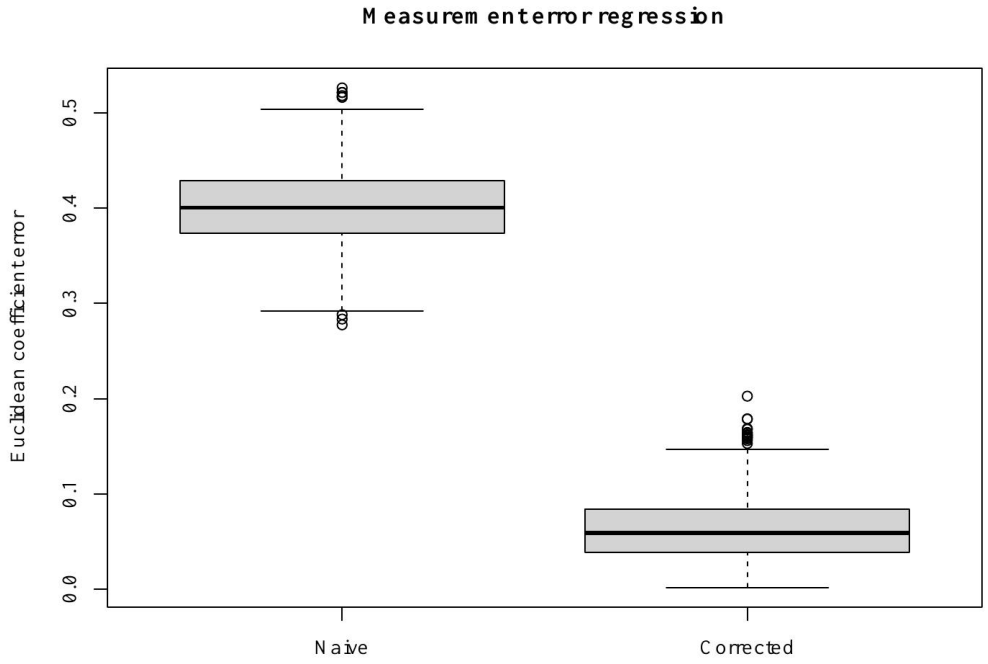


图 3 朴素估计与校正估计的系数欧氏误差

朴素估计的平均误差约为 0.4021，而校正估计的平均误差约为 0.0637；箱线图也显示校正后的误差分布明显更集中。这与经典测量误差导致衰减偏误的结论一致，第二个独立代理变量能够显著降低回归系数向量的估计误差，提高参数估计的准确性。

运行模式	Worker 数	1000 次耗时 秒	相对串行加速比
串行	1	0.190	1.0000
并行	1	0.320	0.5938
并行	2	0.221	0.8597
并行	4	0.176	1.0795

4 个 worker 的耗时最短，但加速比只有 1.0795；2 个 worker 和并行 1 个 worker 反而慢于串行。原因是单次模拟规模较小，多进程调度、数据传输和结果合并的开销占比较高。该结果也说明并行性能取决于任务粒度，不一定随 worker 数增加而线性提高。

4 OpenMP 图像卷积

4.1 并行实现

程序对灰度图像使用 3×3 卷积核，直接使用给定卷积核而不翻转。边界通过 mirrorIndex 进行镜像延拓，计算完成后将输出像素截断到 0 到 1。像素计算公式为

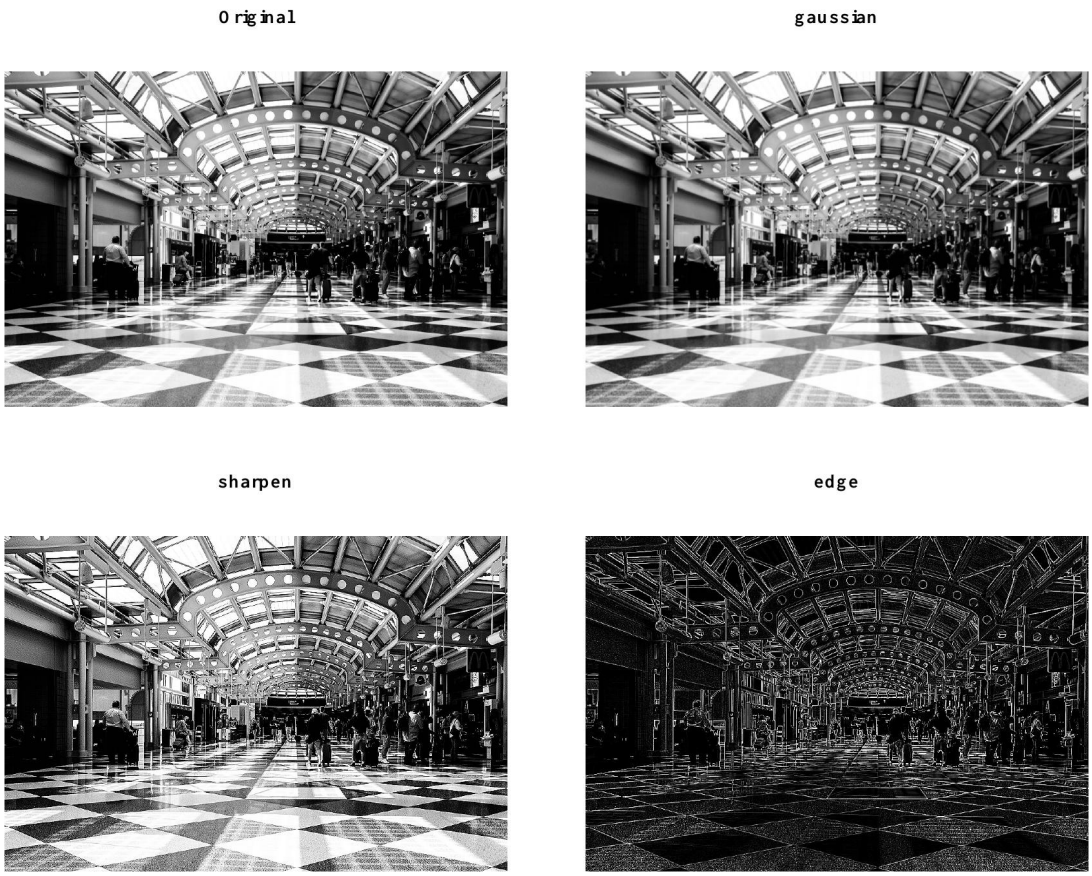
$$C_{ij} = \sum_{a=-1}^{1} \sum_{b=-1}^{1} A_{i+a,j+b} \, K_{a+2,b+2}$$

OpenMP 使用 parallel for 和 schedule(static) 对输出列进行并行划分，并通过 num_threads 按参数设置线程数。输入图像和卷积核只读，每个输出像素只由一个线程写入，因此不存在数据竞争；并行区域中也没有调用 R 的输出、随机数或报错函数。

4.2 正确性检验与图像结果

边界测试包括 1×1、1×3、3×1 和 4×5 四种输入，并分别使用高斯模糊、锐化和边缘检测卷积核。测试线程数为 1、2、4。所有输出尺寸均与输入相同，元素有限且位于 0 到 1；串行与并行结果的最大绝对差小于 10^{-10} 。正式实验将原始 1365×2048 图片按步长 2 缩放为 683×1024。

图 4 原图及高斯模糊、锐化和边缘检测结果



高斯核平滑了局部噪声和细小纹理；锐化核增强了结构边缘与局部对比度；边缘检测核保留了建筑轮廓、地面线条和人物边界。三种结果与卷积核的预期作用一致。

4.3 运行时间

计时使用同一幅 683×1024 灰度图和高斯卷积核，不包含编译、图片读取、绘图和结果验证。measureTime 先预热一次，再自动增加每批调用次数，使批量计时不少于 0.2 秒；每种设置重复 3 批并取中位数。加速比定义为

$$S_p = \frac{T_{\text{serial}}}{T_{\text{parallel},p}}$$

运行模式	线程数	每次耗时 秒	加速比
串行	1	0.010969	1.0000
OpenMP	1	0.012344	0.8886
OpenMP	2	0.006813	1.6101
OpenMP	4	0.004063	2.7000

从实验结果来看，OpenMP 并行计算的运行效率随线程数增加而提高。使用单线程时，并行版本的运行时间略高于串行版本，加速比为 0.8886，主要是由于并行调度存在额外开销。使用 2 个线程时，加速比提高至 1.6101；使用 4 个线程时，加速比达到 2.7000，说明多线程并行能够有效缩短图像卷积的计算时间。不过，加速比并未随线程数增加而等比例提高，这与线程调度及内存访问等额外开销有关。

实验总结

四部分实验分别展示了低秩近似、稀疏矩阵求解、多进程随机模拟和共享内存并行计算。低秩实验中，增大 K 可以持续降低重构误差，但同时增加存储量；高斯过程实验中，较大的 lambdaSq 会产生更平滑但可能欠拟合的曲线；测量误差实验表明，使用第二代理变量进行校正能够显著降低估计误差，而短任务的多进程加速较为有限；OpenMP 卷积在保证串并结果一致的前提下实现了像素级并行。